ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF COMPUTER AND
INFORMATICS

TinyNPU

Graduation Project Final Report

Fırat Kızılboğa
150200115

Department: Computer Engineering
Division: Computer Engineering

Advisor: Assoc. Prof. Dr. Ayse Yilmazer

March 2026

Statement of Authenticity

I/we hereby declare that in this study

1. all the content influenced from external references are cited clearly and in detail,
2. and all the remaining sections, especially the theoretical studies and implemented software/hardware that constitute the fundamental essence of this study is originated by my/our individual authenticity.

İstanbul, Mart 2026

Fırat Kızılboğa

Acknowledgments

I would like to thank my advisor, Assoc. Prof. Dr. Ayse Yilmazer, for her guidance throughout this project. I also thank the people whose open-source tooling and research made it possible to build, analyze, and validate the TinyNPU hardware and software stack.

TinyNPU

(SUMMARY)

TinyNPU is a compact neural-network accelerator built and evaluated as a full hardware/software system. It combines an output-stationary systolic-array NPU with a segmented compiler and runtime, a quantized model-preparation flow, and RTL-backed validation.

The hardware centers on an $N \times N$ systolic array, a unified buffer, an instruction memory, streamer logic, and a post-processing unit that performs bias addition, rescaling, activation, and packed writeback. The software traces PyTorch models, partitions them into accelerator segments and explicit host operations, lowers supported segments into TinyNPU instruction and buffer images, and executes the result on the attached CV32E40P core.

This report treats the whole stack as the useful unit of design. Tensor layout, quantization boundaries, host fallbacks, runtime overhead, and realizable clock frequency all affect whether the accelerator is practical to use. The implemented system is validated on trained MNIST-derived MLP and convolution models, controlled multi-tile GEMM benchmarks, small transformer-like blocks, and synthesis/timing experiments that expose the system limits.

TinyNPU

(ÖZET)

TinyNPU, sistolik dizi tabanlı bir nöral işlemci birimini onu kullanılabilir hale getiren derleyici, çalışma zamanı, niceleme akışı ve benzetim tabanlı doğrulama ortamıyla birlikte ele alan bütünleşik bir donanım-yazılım sistemidir. Amaç yalnızca küçük bir hızlandırıcı çekirdeği tasarlamak değil, bu çekirdeği gerçek bir makine öğrenmesi iş akışında kullanılabilir hale getirmektir. Bu nedenle sistem, PyTorch tabanlı model hazırlama sürecini destekleyen, nicelemeli modelleri hızlandırıcı segmentlerine dönüştüren, desteklenmeyen işlemleri ana işlemci üzerinde çalıştıran ve sonuçları yazılım referanslarıyla doğrulayan bir yapı olarak tasarlanmıştır.

Donanım tarafında TinyNPU, birleşik tampon bellek ve komut belleği ile beslenen sistolik dizi tabanlı bir hesaplama çekirdeği etrafında kurulmuştur. Girdi aktivasyonları ve ağırlıklar birleşik tampondan akıtıcı birimlere, oradan da sistolik diziye iletilir. Hesaplama sonucunda oluşan değerler son işleme biriminden geçirilerek paketlenmiş biçimde tekrar belleğe yazılır. Bu yapı, yoğun nicelemeli matris işlemleri için verimli bir veri yolu sağlarken, dışarıdan görülen programlama modelini de yönetilebilir düzeyde tutmaktadır. Yazılım tarafı ise yüksek seviyeli model işlemlerini hızlandırıcı segmentleri ve gerekirse ana işlemci işlemlerini birlikte içeren bir yürütme planına dönüştürmektedir.

Bu çalışmada sistem, eğitilmiş MNIST tabanlı MLP ve evrişim modelleri, kontrollü GEMM deneyleri, küçük transformer benzeri bloklar ve sentez/zamanlama deneyleri üzerinde doğrulanmıştır. Ayrıca, hızlandırıcının gerçek hesaplama çevrimlerini paketleme, arayüz, çalışma zamanı ve saat frekansı maliyetlerinden ayıran bir ölçüm altyapısı kurulmuştur. Böylece çekirdeğin kendi verimliliği ile entegrasyon maliyetleri ayrı ayrı değerlendirilebilmektedir. Tekrarlı çıkarım senaryoları için oturum tabanlı çalışma ve ağırlık kalıcılığı desteği de eklenmiştir.

Rapor, bu sistemi donanım mimarisi, model hazırlama ve derleme akışı, doğrulama yöntemi ve deneysel sonuçlarıyla birlikte sunmaktadır. Sonuçlar, sistolik çekirdeğin güçlü hesaplama sağladığını, ancak toplam davranışın hâlâ veri yerleşimi, bölütleme ve çalışma zamanı maliyetlerine duyarlı olduğunu göstermektedir.

Contents

1 Introduction and Project Summary

1.1 Background and Motivation

Modern deep neural networks demand substantial compute and memory bandwidth even at inference time. This has made specialized accelerator design a central topic both for datacenter-scale systems and for smaller edge-oriented deployments [8, 4]. Quantization is one of the main techniques that makes such deployment practical, because it reduces memory traffic and allows arithmetic throughput to increase when narrow integer types are used instead of floating-point data.

However, quantization is not a single fixed target. Mixed-precision work has shown that different layers or kernels inside the same model can tolerate different bit-widths [11]. This creates a hardware opportunity and a software problem at the same time. Hardware should benefit from narrower arithmetic when it is safe to do so, but the software stack must still prepare the model, preserve numerical intent, and decide where accelerator execution ends and host execution begins.

1.2 Problem Definition

The main problem addressed by TinyNPU is not only how to build a small systolic-array accelerator, but how to make mixed-precision quantized computation usable from a real model flow. Fixed-precision accelerators leave throughput on the table when different parts of a model tolerate different bit-widths. CPU-centric low-bit approaches, on the other hand, still pay software-side cost for packing, unpacking, and data movement around the arithmetic [3, 1, 10].

This project therefore treats the problem as a co-design task. The hardware must expose a regular packed-integer datapath, and the compiler/runtime must make tensor layout, quantization boundaries, and host fallbacks explicit. The result should be measurable honestly, with accelerator compute, runtime overhead, and host-side work kept separate.

1.3 Project Scope and Approach

TinyNPU approaches this problem with four coupled components:

1. **A parameterized mixed-precision accelerator:** TinyNPU is centered around an output-stationary $N \times N$ systolic array, a unified buffer, streamer logic that feeds the array, and a post-processing unit that performs bias addition, rescaling, activation, saturation, and packed writeback.

2. **Packed integer computation:** The processing elements support packed INT16, INT8, and INT4 execution modes. Throughput therefore scales with precision

reduction, not by changing the global architecture, but by packing more useful work into the same datapath.

3. **A segmented compiler/runtime:** The software flow traces supported PyTorch models, partitions them into accelerator segments and explicit host-side operations, lowers legal accelerator work into instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation.

4. **Verification and measurement infrastructure:** The project includes compiler-owned expected tensors, runtime verification points, repeated-run sessions, and benchmark accounting that separates accelerator compute from protocol and software overhead.

TinyNPU runs trained MNIST-derived MLP and convolution models through the full compiler/runtime stack, benchmarks controlled multi-tile GEMM workloads, and evaluates small GPT/Llama-like blocks to study transformer lowering. These experiments expose both the arithmetic capability of the core and the system-level cost of staging, boundaries, host operations, timing closure, and runtime service.

2 Comparative Literature Survey

2.1 Mixed-Precision Quantization

Mixed-precision quantization work is the clearest algorithmic motivation for TinyNPU. Wu et al. formulate layer-wise quantization as a search problem and show that different layers can be assigned different bit-widths without treating the whole model uniformly [11]. This is the key observation behind a variable-precision accelerator: if model layers do not all need the same precision, a fixed-precision datapath cannot exploit the full efficiency opportunity.

Quantization is also relevant as a compiler concern rather than only a hardware concern. SmoothQuant is an example of pushing difficult numerical adjustments into an offline transformation so that runtime execution becomes simpler and more hardware-friendly [12]. TinyNPU follows the same general philosophy even though it does not implement the full SmoothQuant method itself: model preparation, fused rescaling, and compiler-ready conversion are treated as software responsibilities that simplify the accelerator contract.

2.2 High-Performance Matrix Multiplication

The mathematical core of TinyNPU is still matrix-style computation, so the project is strongly informed by classical GEMM literature. Goto and van de Geijn show that high-performance matrix multiplication depends not only on arithmetic throughput, but on careful tiling and data movement through the memory hierarchy [5]. The same principle appears in TinyNPU at a smaller scale. When matrices exceed the dimensions of the systolic array, the compiler and runtime must tile along the M, N, and K dimensions while preserving the data reuse pattern expected by the array and its post-processing path.

2.3 Existing Accelerator Architectures

On the accelerator side, Google's TPU remains the canonical reference for systolic-array-style matrix acceleration [8]. Gemmini is especially relevant because it demonstrates how a systolic array, programming model, and system context can be treated as one full-stack research object rather than as an isolated RTL block [4]. That full-stack perspective is one of the strongest methodological reference points for TinyNPU.

An educational TinyTPU implementation was a useful starting point for understanding systolic-array organization, but it was not enough for this project [2]. Extending the design toward mixed precision, tiled execution, and explicit post-processing exposed the need for a custom software contract, explicit host/accelerator boundaries, and a compiler/runtime that owns packing and verification.

Taken together, these references define TinyNPU's design space. It is not trying to outscale industrial accelerators. Its goal is a compact mixed-precision accelerator system whose hardware, compiler/runtime, and measurement boundary can be studied together on one working platform.

3 Developed Approach and System Model

The system is described here as four cooperating subsystems: the TinyNPU hardware, the quantizer and model-preparation path, the compiler, and the runtime. This matches the real engineering boundaries of the stack and keeps the movement of tensors, instructions, and packed data explicit.

3.1 System Overview

At a high level, a model starts either as a quantized PyTorch module or as a compiler-ready artifact produced by the quantization path. The compiler traces the model, partitions it into accelerator-supported regions and explicit host operations, lowers legal regions into TinyNPU instruction and buffer images, and packages the result as a compiled artifact. The runtime then executes that artifact step by step. Inside the hardware, the unified buffer, streamers, systolic array, and post-processing unit implement the matrix-style execution path.

The important requirement is a clean contract between logical tensors, physical packed layouts, host-side fallback behavior, and the hardware pipeline. The rest of this section follows that order: hardware first, then model preparation, then compilation, then runtime execution and verification.

3.2 NPU

The TinyNPU hardware is organized around a control path and a compute path. The control path sequences instructions and services the host interface. The compute path consists of the unified buffer, streamer logic, a parameterized $N \times N$ output-stationary systolic array, and the post-processing unit.

The baseline host contract is MMIO. Through the MMIO command path, the host can launch execution, read status, and transfer memory payloads through the memory value register (MMVR) interface one NPU word at a time. That path is portable because it only assumes the standard control registers are present.

When the integration provides it, the same UB and IM storage can also be exposed through a wider shared-SRAM host window. This path is faster because it bypasses the MMIO memory-service loop for bulk preload and readback traffic. The arbitration policy is intentionally simple: it is time-multiplexed rather than fully concurrent. The host may use the shared window while the accelerator is idle, and shared access is blocked while the NPU is busy. This avoids adding a more complex concurrent memory arbiter while still capturing the main performance benefit of direct SRAM access.

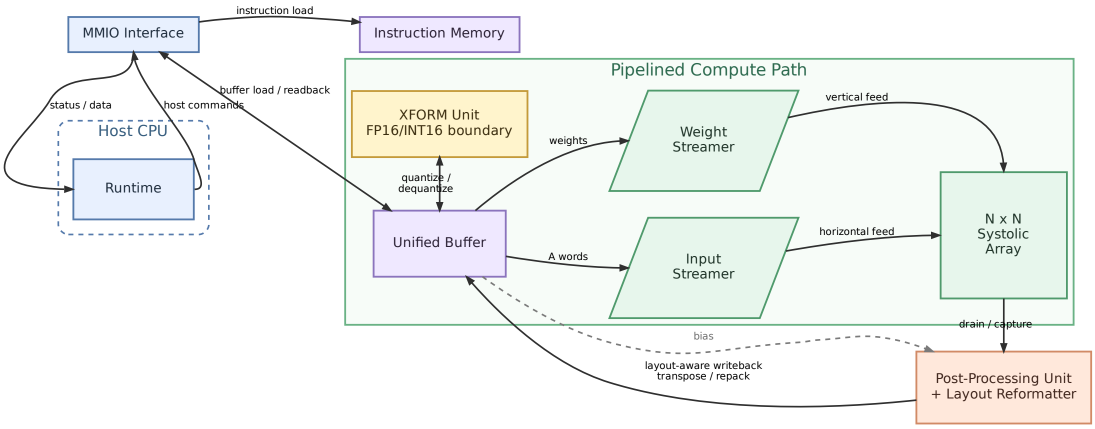

Figure 3.1: Top-level TinyNPU hardware architecture.

Figure 3.1 shows the top-level organization. The compute path is a matrix path: the unified buffer feeds activation and weight streamers, the streamers feed the systolic array, and drained accumulators pass through the PPU before returning to the buffer. Boundary quantization and dequantization are handled by the runtime/compiler contract rather than by a separate hardware transform instruction.

3.2.1 Unified Buffer and Physical Roles

The unified buffer is the central staging memory of the datapath. Architecturally it provides one activation-side read path, one weight-side read path, and a masked write path for packed output updates. This organization keeps both feed directions active at the same time and leaves room for a banked implementation rather than forcing one monolithic memory block.

Within that buffer, the names A, B, C, and BIAS refer to physical roles in the datapath. A is the left-fed operand: its tiles are packed so that each unified-buffer row can launch one horizontal activation vector into the array boundary. B is the top-fed operand: its tiles are packed for the vertical weight stream. C is the normal output layout: it stores quantized output tiles for host readback, final model outputs, verification, and any result that does not need to be consumed immediately as the next A-side operand. BIAS is a separate region consumed only by the PPU. These names therefore describe how data is stored and consumed by the hardware pipeline, not whether a tensor was logically an activation, a weight, or an intermediate result in the original model.

The physical layouts are intentionally different. A layout is feed-oriented: each unified-buffer word represents one activation-side feed step, with one 16-bit lane per array row and one, two, or four packed K-elements per lane for INT16, INT8, or INT4. B layout is similar on the weight side, with each word representing one top-fed step into the array columns. C layout is writeback-oriented instead: it stores output tiles compactly by output column, and at lower precision it packs several logical results into subword slots. C is still useful as the default result layout, but it is not automatically a good input layout for the next layer. When a result must feed another NPU matmul without host layout repair, the compiler selects an A-compatible writeback layout instead.

3.2.2 Systolic Array and Skewers

The array is output-stationary: packed activation words enter from the left edge and move one PE to the right per cycle, packed weight words enter from the top edge and move one PE downward per cycle, and each PE keeps its output accumulator stationary for the full reduction along the K dimension. Only after the last K contribution has been consumed does the array switch into drain mode and shift those accumulated results downward into the post-processing unit. This choice fits the dominant tiled-GEMM workload well because it keeps the reduction local to the PE and avoids repeatedly moving partial sums through the buffer system during the K loop.

The streamer logic between the unified buffer and the array provides the timing skew that makes the systolic wavefront legal. On the activation side, row i is delayed before it enters the left edge; on the weight side, column j is delayed before it enters the top edge. The values themselves do not change. What changes is when they enter the array. That timing offset creates the diagonal wavefront expected by the schedule, so the activation and weight pieces that belong together meet at the correct PE in the correct cycle. A tile therefore executes in three phases: feed the packed K-steps into the array, allow the wavefront to fill across the remaining diagonals, and then drain the stationary accumulators.

3.2.3 Processing Element

The processing element is where mixed precision becomes concrete. Each PE consumes one packed activation word and one packed weight word. In INT16 mode that produces one signed multiply, in INT8 mode two byte multiplies, and in INT4 mode four nibble multiplies. The products are summed and accumulated into a 64-bit stationary register.

One practical implementation detail is that the vertical PE-to-PE link is wider than the packed weight word used during feed. The reason is reuse. During the compute phase the array only needs the packed incoming weight value, but during drain the same physical column path is reused to move full accumulators toward the PPU. Reusing one wider column link keeps the PE interface uniform across compute and drain instead of introducing a second dedicated drain network.

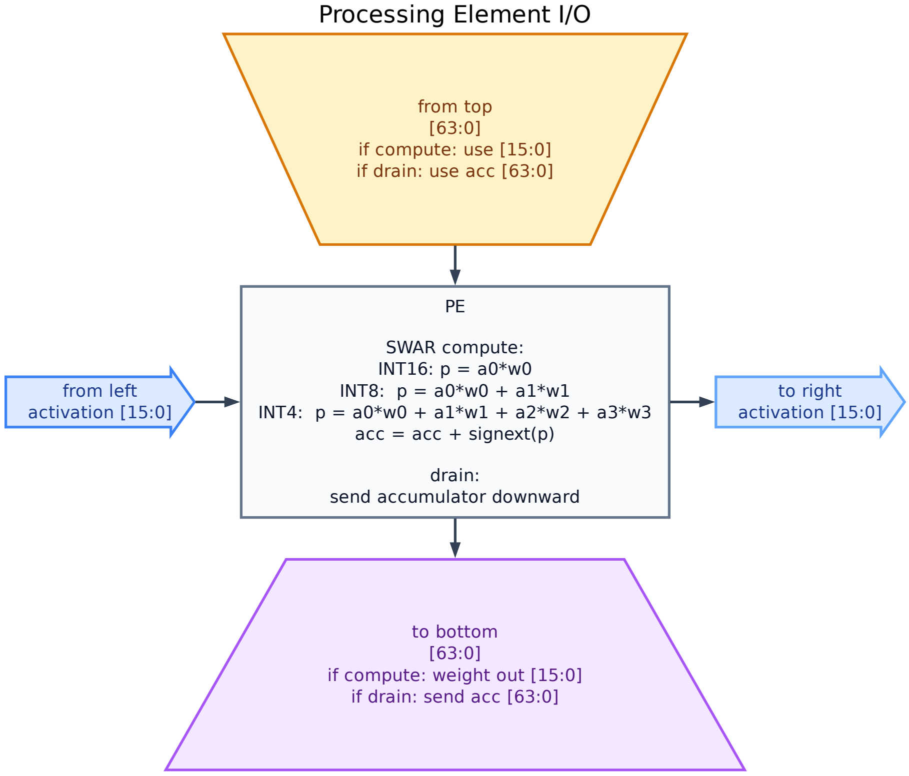

Figure 3.2: Processing-element I/O in compute and drain modes.

3.2.4 Post-Processing Unit

The post-processing unit (PPU) is an N-lane row processor backed by an internal $N \times N$ tile buffer. During drain, it receives one accumulator row vector per cycle from the bottom of the systolic array, processes the row in parallel, and stores the quantized result into its internal buffer. After the full tile has been captured, it writes the buffered rows back to the unified buffer.

Before drain begins, the PPU loads the 32-bit bias vector for the output columns. For each drained row it then evaluates the same requantization pipeline lane by lane:

$$q = \mathrm{clip}_p\left(\mathrm{act}\left(\left((acc + bias) \times multiplier + 2^{shift-1}\right) \gg shift\right)\right)$$

where acc is the 64-bit drained accumulator, $bias$ is a sign-extended 32-bit bias term, $multiplier$ is the 16-bit fixed-point requantization factor, $\gg$ is an arithmetic right shift, $\mathrm{act}(\cdot)$ selects the configured post-processing nonlinearity, and $\mathrm{clip}_p(\cdot)$ saturates to the signed range of the selected output precision. The compiler enforces the hardware ABI for this path: matmul rescale shifts are limited to 0–63 and the hard-GELU input-domain shift is limited to 0–15. The activation stage supports plain ReLU, an integer sigmoid path derived from a DI-Exp-style integer nonlinear primitive in the spirit of I-LLM [7], and a hardware-friendly hard-GELU approximation used when GELU is lowered into the accelerator [6, 9]. Concretely, the datapath performs bias addition, fixed-point multiplication, round-to-nearest when $shift > 0$, the selected activation, and final saturation to INT16, INT8, or INT4.

Writeback is also precision-dependent. After the tile buffer has been filled, the ordinary C-layout path emits one buffered row per writeback cycle. For INT16 output, each 16-bit lane is written back directly. For INT8 and INT4 output, the PPU shifts the quantized value into the byte or nibble slot selected by the packed write offset, and the unified buffer applies a bit mask so that only that subword is updated while previously written sibling tiles in the same physical word are preserved. The control unit derives this packed write offset from the output tile index: two logical M-tiles share one physical word for INT8, and four logical M-tiles share one physical word for INT4.

The PPU therefore supports several practical writeback forms. For host-visible outputs it emits the ordinary C-oriented packed result layout. For internal chained tensors it can instead emit an A-compatible layout so the next accelerator consumer can read the result directly without a host-side layout conversion. The RTL also contains INT16 K-cache and V-cache append modes used by transformer-shaped experiments. The compiler selects the required form when it lowers a segment.

3.2.5 Control and Instruction Format

This behavior is programmed through a fixed-width 256-bit instruction format in instruction memory. The `MATMUL` instruction encodes base addresses, tile counts, PPU configuration, output-layout selection, cache writeback modes, and B-side read modes. The control unit therefore fetches one coarse-grained instruction for a tiled matrix-multiplication region and then steps through clear, feed, wait, drain, bias-load, and writeback states internally.

Table 3.1: Implemented `MATMUL` instruction field layout in the RTL.

Bits	Field	Meaning
[255 : 252]	`opcode`	Operation code; `ISA_OP_MATMUL` selects the tiled matrix-multiplication sequencer.
[251 : 248]	`writeback_mode`	Selects ordinary writeback or INT16 K/V cache append writeback.
[247 : 232]	`A_base`	Unified-buffer base address of A tiles in activation layout.
[231 : 216]	`B_base`	Unified-buffer base address of B tiles in weight layout.
[215 : 200]	`C_base`	Unified-buffer base address of C writeback tiles.
[199 : 184]	`output_word_offset`	Word offset added to the C writeback base for segmented output placement.
[183 : 168]	`M_tiles`	Number of output-tile positions along the M dimension.
[167 : 152]	`K_tiles`	Number of reduction passes per output tile.
[151 : 136]	`N_tiles`	Number of output-tile positions along the N dimension.
[135 : 120]	`BIAS_base`	Unified-buffer base address of the bias payload consumed by the PPU.
[119 : 112]	`shift`	Arithmetic right-shift amount for requantization.
[111 : 96]	`multiplier`	Fixed-point requantization multiplier applied in the PPU.
[95 : 88]	`activation`	Post-processing activation selector consumed by the PPU; software uses it for ReLU, integer sigmoid, and hard-GELU modes.
[87 : 86]	`out_precision`	Output packing precision used by the PPU writeback path.
[85 : 84]	`write_offset`	Reserved packed-lane field in the instruction format. The effective write offset is derived internally from output-tile index and output precision during packed writeback.
[83 : 82]	`in_precision`	Input precision mode broadcast to the systolic array PEs.
[81 : 74]	`h_gelu_x_scale_shift`	Per-op hard-GELU input-domain shift used by the PPU when GELU is lowered into the accelerator.
[73 : 72]	`output_layout`	Physical writeback layout selector. The RTL supports ordinary C-style output packing plus an A-compatible transposed/repacked form for chained matmuls.
[71 : 56]	`b_word_offset`	Word offset added to the B operand base for segmented or cached B-side reads.
[55 : 52]	`b_read_mode`	Selects ordinary B reads or INT16 K-cache reads for attention-style workloads.
[51 : 0]	reserved	Unused in the `MATMUL` encoding.

`HALT` terminates the instruction stream and places the control unit in the halted state, `NOP` advances without side effects, and `MOVE` is reserved for internal data movement. In the JIT flow, `MATMUL` and `HALT` are the important emitted hardware operations.

Table 3.2: Implemented instruction opcodes in the RTL control path.

Opcode	Name	Role
`0x0`	`NOP`	No operation; decode advances without performing accelerator work.
`0x1`	`HALT`	Terminates execution and leaves the controller in the halted state until the host issues another command.
`0x2`	`MATMUL`	Coarse-grained tiled matrix-multiplication instruction with integrated PPU configuration.
`0x3`	`MOVE`	Internal data-movement operation reserved by the RTL control path.

Instruction execution is paired with a small MMIO command interface on the host side. The host does not directly drive internal control states; instead it writes a command register to request memory service or to start execution from a given instruction-memory address. In the general contract, MMIO/MMVR is sufficient to operate the accelerator by itself. When the platform also provides the shared-SRAM mapping described above, that faster path is used for bulk preload and readback while MMIO remains the universal control and fallback interface.

Table 3.3: Host-visible MMIO commands used to service the accelerator.

Command	Value	Meaning
`CMD_WRITE_MEM`	`0x01`	Writes the MMIO data register value into unified-buffer or instruction-memory space at the selected address.
`CMD_READ_MEM`	`0x02`	Reads the selected memory location back into the MMIO-visible data register.
`CMD_RUN`	`0x03`	Starts instruction execution from the instruction-memory address carried in the argument register.

3.3 Quantizer and Model Preparation

The quantization path exists to prepare trained models into coherent compiler-ready artifacts. Its goal is practical rather than open-ended: make weights, activations, ranges, signedness, and mixed-precision choices explicit enough that the compiler can lower the model without losing numerical coherence.

This subsystem also includes the mixed-precision helper. It is not a global bit-allocation solver. It provides a one-layer-at-a-time sensitivity workflow that can generate usable mixed-precision configurations, recalibrate them, and pass them to the compiler-ready conversion path.

The evaluated path uses post-training quantization for trained MNIST-derived models. Weights are quantized into the selected integer precision, activation scales are fixed by the compiler-ready configuration, and 32-bit bias terms are kept explicit for the PPU. This is a narrower claim than a complete global mixed-precision solver, but it is sufficient to test whether the compiler/runtime and RTL execute trained weights without losing the intended classification behavior. Table 3.4 reports the trained-model accuracy checks.

Table 3.4: Trained MNIST `is_zero` accuracy after post-training quantization. Balanced accuracy uses an equal zero/nonzero test split; full-test accuracy uses the original MNIST test distribution.

Model	FP32 balanced	FP32 full	INT16 PTQ	INT8 PTQ	INT4 PTQ
Conv wide32	98.16%	98.67%	98.16%	98.16%	98.11%
MLP h256	98.57%	98.33%	98.57%	98.57%	98.37%

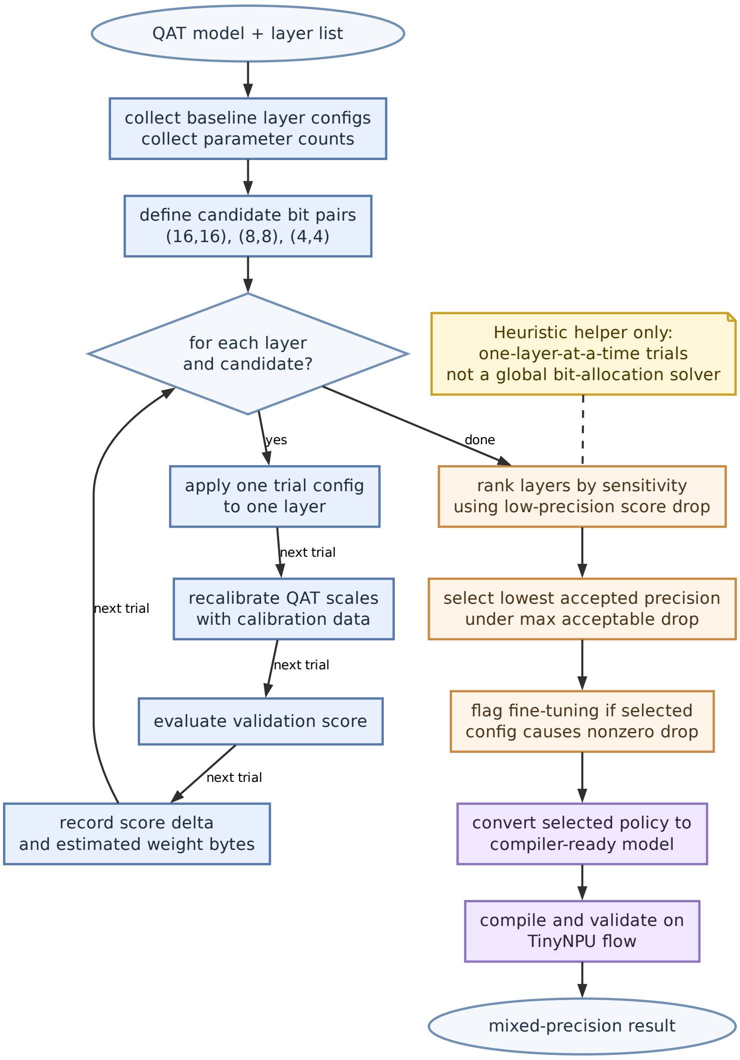

Figure 3.3: Heuristic mixed-precision selection flow used to obtain compiler-ready model variants.

The role of this subsystem is to bridge trained or quantized models into forms that the TinyNPU compiler can lower without losing track of numerical meaning.

3.4 Compiler

The compiler converts model structure into a mixed execution structure. Rather than generating a monolithic accelerator-only program, it constructs an `ExecutionPlan` whose steps are typed as `NpuSegment`, `HostOp`, or `VerifyTensor`. This keeps unsupported behavior and verification points explicit instead of hiding them inside an accelerator-only view.

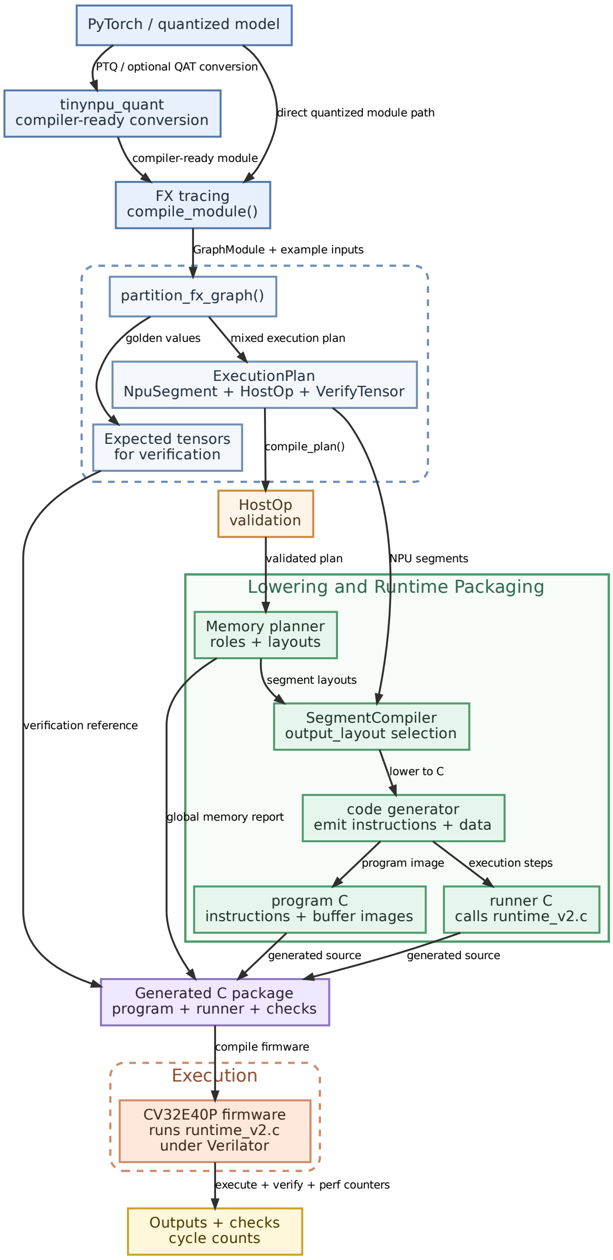

Figure 3.4: Compiler and runtime flow used by the TinyNPU JIT path.

Figure 3.4 summarizes the software flow. A model enters either directly as a supported quantized PyTorch module or through a compiler-ready conversion path built around `tinynpu_quant`. The frontend traces the model, partitions it into a mixed execution plan, validates host operations, and lowers each accelerator segment through the memory planner and `SegmentCompiler`. The code generator then emits two pieces of C: a program file containing NPU instructions and buffer images, and a runner file that calls `runtime_v2.c` with the required inputs, execution steps, and verification data.

Packing is part of the compiler contract as well. The backend uses role-specific physical layouts for activation-side tensors, weight-side tensors, output buffers, and bias payloads. These are physical storage forms rather than semantic tensor meanings, but they are essential because legal execution depends on matching memory layout to the real feed directions and writeback format of the NPU. Internal NPU outputs are not assumed to be consumable by a following A-side input when they are written in ordinary C-style output layout. The compiler annotates the required physical output layout explicitly, and the emitted instruction stream carries that selection into the RTL writeback path.

The important compiler policy is not the cycle-by-cycle tile schedule but where accelerator regions begin and end. The partitioner emits a mixed execution plan and forces a segment boundary whenever execution must return to the host, a verification point is requested, or an operation is outside the supported NPU contract. As a result, host-side work such as dequantization, softmax, mean, layout restoration, and verification is represented directly as `HostOp` or `VerifyTensor` steps rather than being buried inside an accelerator launch.

Memory planning makes those execution boundaries explicit in the buffer layout. The unified buffer is divided into a static zone and a dynamic zone: weights and biases receive globally assigned packed addresses that can be shared across segments, while activations and outputs are placed in a per-segment dynamic region with liveness-based reuse. The planner also treats layout as a real physical requirement. If a tensor must be emitted in an A-compatible layout for an internal chained consumer, that requirement is carried explicitly instead of being inferred informally.

3.5 Runtime and Verification

The deployed runtime is the C file `runtime_v2.c` linked into the generated CV32E40P firmware. It executes the compiled plan as a mixed CPU/NPU loop. For each step, it either launches an NPU segment, executes a required CPU fallback operation, or checks an expected tensor. CPU work is therefore visible both in the plan and in the benchmark accounting.

`runtime_v2.c` also owns repeated-run behavior. It can preserve reusable state such as static unified-buffer contents and preloaded instruction-memory images across runs.

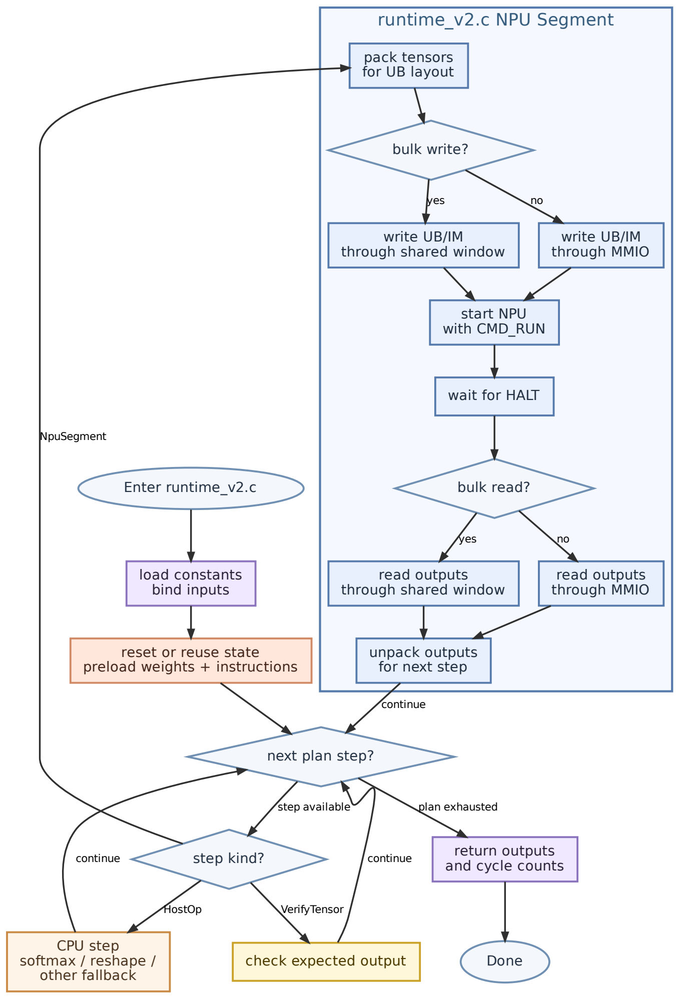

Figure 3.5: `runtime_v2.c` flow for executing a compiled TinyNPU artifact.

Figure 3.5 shows this firmware loop. For accelerator segments, `runtime_v2.c` packs logical tensors into the physical layouts expected by the datapath, stages data into the unified buffer and instruction memory, starts the NPU, waits for completion, and reads the outputs back. Output-layout support lets chained internal tensors be written directly in the layout needed by the next accelerator consumer.

Verification is built into this same path. Compiler-owned expected tensors can be checked during execution, and the runtime can record RTL cycle counts for accelerator run windows separately from software-side overhead.

3.6 GPT- and Llama-Like Blocks

The transformer experiments use reduced-dimensional blocks that preserve the execution structure of GPT-style and Llama-style decoder layers without claiming full pretrained-model compatibility. They are useful because they exercise the parts of the stack that simple MLP and convolution tests do not: normalization, attention score formation, softmax, key/value cache handling, residual connections, and feed-forward sublayers.

The Llama-like block uses RMSNorm, grouped-query attention, RoPE on Q/K, an attention output projection, and a SwiGLU feed-forward tail. Dense projections are lowered to INT16 NPU matmuls: Q/K/V projection, QK score matmul, probability-times-value matmul, attention output projection, gate/up projections, and down projection. The CPU handles the non-matrix pieces: RMSNorm, RoPE, softmax, residual adds, SiLU, elementwise feed-forward multiplication, and the quantize/dequantize boundaries around those operations. Prefill materializes the key/value cache for the prompt. Decode reuses that cache, appends the new K/V entries, and attends over the prompt plus the decode token.

The GPT-2-like block uses LayerNorm, causal self-attention, an attention output projection, residual connections, and a two-layer feed-forward network. Its NPU work is the same class of dense matmul work: Q/K/V projections, score and value matmuls, output projection, and feed-forward projections. The CPU handles LayerNorm, softmax, residual adds, and the non-matrix attention bookkeeping. The reported GPT-2-like experiment chains two blocks back-to-back and includes prefill, decode, and cache reuse so that the runtime is forced to carry intermediate state across more than one block.

Table 3.5: Transformer-like block lowering boundary. The blocks are reduced-size structural tests, not full pretrained GPT-2 or Llama models.

Block	**NPU work**	**CPU/runtime work**
Llama-like	INT16 Q/K/V projections, attention score matmul, probability-times-value matmul, attention output projection, gate/up projections, down projection	RMSNorm, RoPE, softmax, residual adds, SiLU and elementwise feed-forward multiply, K/V cache scatter/append, quantize/dequantize boundaries
GPT-2-like	INT16 Q/K/V projections, attention score matmul, probability-times-value matmul, attention output projection, feed-forward projections	LayerNorm, causal mask and softmax, residual adds, cache reuse across prefill and decode, quantize/dequantize boundaries

3.7 Scope Boundaries

The system targets a narrow but concrete contract. The mixed-precision helper produces compiler-ready configurations rather than solving the global bit-allocation problem. The compiler supports a bounded operator set and routes remaining work through host steps. The evaluation focuses on trained MNIST-derived models, controlled kernel benchmarks, and small transformer-like blocks, which is enough to study the interaction between architecture, compilation, and runtime without overstating workload coverage.

4 Evaluation Setup

All reported performance numbers come from the CV32E40P example testbench with TinyNPU attached in Verilator. The core controls TinyNPU through MMIO command and status registers. The integration also provides a shared-SRAM host window for unified-buffer and instruction-memory traffic. In this setup the shared window is an idle-time preload and readback path: the host uses it while the accelerator is idle, then execution transfers ownership to the accelerator for the run itself.

The evaluation uses three views. The first is a controlled multi-tile GEMM benchmark used to study end-to-end MAC/cycle under the shared-SRAM runtime path. The second is a trained-model comparison against a CPU/ONNX-style baseline, converted to wall-clock time using measured timing estimates. The third is a small transformer-like block study that checks whether GPT/Llama-shaped execution improves as model dimensions grow.

Correctness for the measured runs comes from compiler-owned expected tensors, runtime verification, or `autoverify` checks when explicit verify steps are not emitted. Separate unit, regression, and cocotb tests are used during development, but the headline numbers in this section come from the CV32E40P+TinyNPU execution path rather than host-only emulation.

Wall-clock comparisons use the Vivado timing boundary. The CPU-only CV32E40P configuration routes at 57.1 MHz. After removing the hardware XFORM path, the CPU+NPU real-BRAM configuration still does not close at 50 MHz; its measured 50 MHz timing violation corresponds to an approximate 33.9 MHz operating point. This is intentionally conservative for the integrated path: because the NPU clock is lower, CPU+NPU must achieve more than 1.68× cycle reduction before it wins in wall-clock time.

Table 4.1: Vivado timing boundary used for wall-clock comparisons. Target device is `xc7a200tsbg484-1`.

Design	Target	WNS	Status	Use in report
CPU only	57.1 MHz	0.000 ns	Meets	CPU wall-clock baseline.
NPU only, real BRAM	50 MHz	-9.851 ns	Fails	Approx. 33.5 MHz; critical path is control/address logic into UB write-mask logic.
CPU+NPU, real BRAM	50 MHz	-9.486 ns	Fails	Approx. 33.9 MHz; used for CPU+NPU wall-clock estimates.
CPU+NPU, abstract memory	50 MHz	+7.359 ns	Meets	Sanity check only; not used as a real implementation result.

5 Comparative Evaluation and Discussion

This section focuses on artifacts that are representative of the implemented system: controlled GEMM scaling, trained-model wall-clock comparison, and small transformer-like block scaling. The main distinction is between inner-kernel capability and system-level behavior. A compute-only speedup is not enough if staging, host operations, launch overhead, or a lower integrated clock dominate the full program.

5.1 Kernel-Level Benchmark

The GEMM scaling benchmark uses one explicit cold-invocation boundary. The numerator is the algorithmic GEMM work, $M \cdot K \cdot N$ logical MACs. The denominator includes static UB/IM preload through the shared-SRAM window plus one full accelerator invocation, including launch, execution, and final INT16 readback. The `autoverify` pass runs after the measured window and is therefore excluded.

To keep this benchmark focused on the accelerator path instead of host-side generic A-packing, the left-hand-side matrix is emitted as compile-time data and prepacked into the static UB image. All rows therefore use the same runtime path, the same INT16 output boundary, and the same cold single-invocation measurement. For the 8×8 array, the peak issue rates are 64 INT16 MACs/cycle, 128 INT8 MACs/cycle, and 256 INT4 MACs/cycle. The last column reports achieved end-to-end MAC/cycle as a fraction of that precision-specific peak.

Table 5.1: Cold end-to-end GEMM scaling. Logical MACs are algorithmic $M \cdot K \cdot N$; the cycle count includes static preload plus one accelerator invocation from the CV32E40P run, with static-prepacked LHS and a fixed INT16 output boundary.

Workload	Prec.	Logical MACs	Peak MAC/cycle	Cold E2E cycles	E2E MAC/cycle	Peak frac.
64^3	INT16	262,144	64	65,665	3.99	6.2%
64^3	INT8	262,144	128	55,422	4.73	3.7%
64^3	INT4	262,144	256	50,304	5.21	2.0%
$96 \times 64 \times 96$	INT16	589,824	64	134,861	4.37	6.8%
$96 \times 64 \times 96$	INT8	589,824	128	117,960	5.00	3.9%
$96 \times 64 \times 96$	INT4	589,824	256	109,513	5.39	2.1%
128^3	INT16	2,097,152	64	277,560	7.56	11.8%
128^3	INT8	2,097,152	128	228,405	9.18	7.2%
128^3	INT4	2,097,152	256	203,831	10.29	4.0%

Table 5.1 shows the effect of moving the benchmark onto a full cold runtime boundary. Once the left-hand-side matrix is prepacked into the static UB image, the precision trend is monotonic but far from the ideal 2× per precision step. TinyNPU reaches only 1.14–1.22× for INT8/INT16 and 1.08–1.12× for INT4/INT8 across these three shapes. That smaller gain is expected here, because preload, launch, and readback remain in the denominator while the arithmetic width changes only the useful work retired per cycle.

Larger kernels still improve achieved throughput because the fixed cold-invocation costs are amortized over more work. The 64^3 row reaches 3.99, 4.73, and 5.21 E2E MACs/cycle for INT16, INT8, and INT4. By 128^3, those values rise to 7.56, 9.18, and 10.29 E2E MACs/cycle. The important point is not a peak-style throughput claim; it is that once full preload and invocation cost are included, lower precision still helps, but much less dramatically than a compute-only metric would suggest.

5.2 Trained Model Wall-Clock Comparison

Table 5.2 compares trained MNIST-derived MLP and convolution models using wall-clock time rather than only cycle ratios. The CPU/ONNX column is a generated CPU baseline. The CPU+NPU column is the hybrid runtime using TinyNPU for supported matrix work and the host for unsupported work such as convolution window materialization. "Cold" includes static preload cost. "Warm" reuses resident weights and instruction images.

Table 5.2: Trained-model wall-clock comparison. CPU/ONNX uses 57.1 MHz; CPU+NPU uses the 33.9 MHz integrated timing estimate.

Workload	CPU/ONNX cycles	CPU+NPU cycles	CPU wall	CPU+NPU wall	Wall speedup
Conv 16ch cold	259,688	124,838	4,548.0 us	3,681.0 us	1.24×
Conv 16ch warm	259,688	113,721	4,548.0 us	3,353.2 us	1.36×
Conv wide32 cold	1,013,314	253,018	17,746.3 us	7,460.5 us	2.38×
Conv wide32 warm	1,013,314	212,717	17,746.3 us	6,272.2 us	2.83×
MLP h256 cold	1,662,361	443,664	29,113.2 us	13,081.9 us	2.23×
MLP h256 warm	1,662,361	45,987	29,113.2 us	1,356.0 us	21.47×

The MLP case shows the best behavior because it is matrix-native and benefits from resident execution. The warm MLP result reaches 21.47× wall-clock speedup even after applying the lower CPU+NPU timing estimate. The convolution case also improves with scale, but it remains more sensitive to host work because the compiler lowers convolution through `im2col`. The wide32 convolution reaches 2.83× warm wall-clock speedup, while the smaller 16-channel case is only 1.36× warm.

5.3 Transformer-Like Block Scaling

Small transformer-like blocks are included to test the compiler/runtime boundary for attention-style workloads. They are not presented as a full language-model throughput claim. The useful question is whether the hybrid path begins to win as dimensions grow and fixed launch/host costs are amortized over more matrix work. In Table 5.3, d is hidden size, h is per-head width, n_h is the number of query heads, n_{kv} is the number of key/value heads, f is feed-forward hidden size, and T is prompt/cache length.

Table 5.3: Small transformer-like runtime comparison. CPU+NPU wall speedups use the 57.1 MHz ONNX2C CPU and 33.9 MHz CPU+NPU timing assumptions.

Workload	ONNX2C CPU cycles	CPU+NPU hot cycles	CPU+NPU cold cycles	Hot wall speedup
Llama-like decode $d = 32, h = 8, n_h = 4, n_{kv} = 2, f = 32, T = 8$	67,902	227,508	241,807	0.18×
Llama-like decode $d = 64, h = 16, n_h = 4, n_{kv} = 2, f = 64, T = 8$	231,279	346,518	398,705	0.40×
Llama-like decode $d = 96, h = 16, n_h = 6, n_{kv} = 3, f = 96, T = 8$	494,338	503,899	618,806	0.58×
Llama-like decode $d = 128, h = 16, n_h = 8, n_{kv} = 4, f = 128, T = 8$	857,577	660,124	862,327	0.77×
Llama-like decode $d = 192, h = 16, n_h = 12, n_{kv} = 6, f = 192, T = 8$	1,876,388	977,033	1,427,556	1.14×

The trend is consistent with the trained-model results, but the stronger ONNX2C baseline and lower integrated clock make the crossover later. Very small decode shapes are not a good fit because host operations and segment overhead consume too much of the total time. In the current no-XFORM path, the Llama-like decode experiment first beats the external ONNX2C CPU baseline at the largest measured resident case, $d = 192$, where hot execution reaches 1.14× wall-clock speedup. Cold one-shot execution at the same shape still loses because static preload remains in the measured window.

5.4 Bottlenecks and Interpretation

The bottleneck ranking is different for each class of workload. Matrix-native MLPs are dominated by whether weights and instruction images are resident. Convolutions are dominated by the remaining host-side `im2col` boundary. Transformer-like blocks are dominated by host operations, segment overhead, and whether the matrix dimensions are large enough to amortize fixed launch costs.

The timing result also matters. On cycles alone, CPU+NPU often looks better than it does in wall-clock time. After applying 57.1 MHz for CPU-only and 33.9 MHz for CPU+NPU, the hybrid path must save enough cycles to overcome the lower integrated clock. The results still show useful wins for resident MLPs, larger convolutions, and the largest measured resident Llama-like decode block, but they also show that timing closure and memory implementation are not secondary details. They directly affect the architectural conclusion.

6 Conclusion and Future Work

TinyNPU is a working mixed-precision accelerator platform with a usable hardware architecture, a PyTorch-oriented model-preparation flow, and a segmented compiler/runtime that keeps host boundaries explicit.

The results show a clear split between compute capability and integration cost. The systolic core delivers useful throughput on tiled GEMM, and the runtime shows wall-clock wins on resident MLPs, larger convolution models, and the largest measured resident Llama-like decode block. At the same time, practical execution still pays heavily for staging, packing, segmentation, host-visible boundaries, and timing closure. The convolution results make this especially clear: the NPU accelerates the matrix work, but the system still pays host-side `im2col`.

The immediate priorities are to close timing on the full CPU+NPU design with realistic memories, reduce avoidable runtime/interface overhead, and increase the size of useful accelerator regions. On the compiler side, the main target is to keep scale assignment, tensor layout, host fallbacks, and cache writeback policies explicit enough that new DNN blocks can be lowered without hand-specialized debugging. On the hardware side, the main target is not adding more features, but making the existing PPU, memory, and control paths timing-clean enough for credible implementation claims.

A Implemented Activation Algorithms

This appendix records the activation functions as they were actually implemented in the TinyNPU software golden path and mirrored in the RTL primitives. These algorithms therefore describe the deployed integer contract rather than an idealized floating-point reference.

Require: Integer input x_{in}, integer parameters $m_I > 0$, $k_I \geq 0$
Ensure: Integer approximation of an exponential-like decay

1: $m_F \leftarrow m_I + (m_I \gg 1) - (m_I \gg 4)$
2: $s_I \leftarrow \left(2^{k_I} + \lfloor m_F/2 \rfloor\right) / m_F$
3: $t \leftarrow -s_I$
4: $q \leftarrow \lfloor x_{in}/t \rfloor$
5: $r \leftarrow x_{in} - q \cdot t$
6: $u \leftarrow (r \gg 1) - t$
7: $y \leftarrow u \gg q$
8: **return** $\max(0, y)$

Algorithm A.1: TinyNPU DI-Exp

Require: Integer input x_{in}, integer parameters m_I, k_I, output precision parameter p_{out}, optional smoothing factor α_{smooth}
Ensure: Integer sigmoid approximation in a signed p_{out}-bit probability domain

$x_s \leftarrow \lfloor x_{in}/\alpha_{smooth} \rfloor$
$e_0 \leftarrow \text{DI-}\textsc{Exp}(0, m_I, k_I)$
if $x_s \geq 0$ **then**
  $e \leftarrow \text{DI-}\textsc{Exp}(-x_s, m_I, k_I)$
  $n \leftarrow e_0$
else
  $e \leftarrow \text{DI-}\textsc{Exp}(x_s, m_I, k_I)$
  $n \leftarrow e$
end if
$d \leftarrow e_0 + e$
$q_{max} \leftarrow 2^{p_{out}-1} - 1$
return $(n \cdot q_{max} + \lfloor d/2 \rfloor) \,/\, d$

Algorithm A.2: TinyNPU DI-Sigmoid

Require: Signed integer input x_{in}, activation-domain shift k_x, slope parameters (n_s, k_s) approximating $1.702 \approx n_s/2^{k_s}$
Ensure: Signed integer hard-GELU result in the same activation domain

$S \leftarrow 2^{k_x}$
$t_3 \leftarrow 3S$
$t_6 \leftarrow 6S$
$s \leftarrow \textsc{RoundShiftSigned}(x_{in} \cdot n_s, k_s)$
$g \leftarrow \min(\max(s + t_3, 0), t_6)$
$y \leftarrow \textsc{RoundDivSigned}(x_{in} \cdot g, t_6)$
return y

Algorithm A.3: TinyNPU Hard-GELU

References

[1] Georgios Armeniakos, Francesco Maria Ottavi, Angelo Garofalo, and Luca Benini. Mixed-precision support for RISC-V soft-SIMD extensions, 2024.

[2] Xander Chin, Kenny Guo, Evan Lin, and Surya Sure. Tiny-tpu: the why and how. https://www.tinytpu.com/, August 2025. Accessed March 20, 2026.

[3] Angelo Garofalo, Francesco Conti, Giuseppe Tagliavini, Davide Rossi, and Luca Benini. XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based iot end nodes. *IEEE Transactions on Emerging Topics in Computing*, 10(4):1849–1863, 2021.

[4] Hasan Genc, Ameer Haj-Ali, Vivek Ittycheriah, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. *Proceedings of the 58th Annual Design Automation Conference (DAC)*, pages 769–774, 2021.

[5] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):12:1–12:25, 2008.

[6] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2016.

[7] Xing Hu, Yuan Cheng, Dawei Yang, Zhihang Yuan, Jiangyong Yu, Chen Xu, and Sifan Zhou. I-llm: Efficient integer-only inference for fully-quantized low-bit large language models, 2024.

[8] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.

[9] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. I-bert: Integer-only bert quantization. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 5506–5518, 2021.

[10] Enrico Reggiani, Alessandro Pappalardo, Max Doblas, Miquel Moreto, Mauro Olivieri, Osman Sabri Unsal, and Adrian Cristal. Mix-GEMM: An efficient hw-sw architecture for mixed-precision quantized deep neural networks inference on edge devices. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[11] Bichen Wu, Yanghan Wang, Peizhao Zhang, Yonglong Tian, Peter Vajda, and Kurt Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search, 2018.

[12] Guangxuan Xiao, Ji Lin, Meryem Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 38087–38099, 2023.